

Venkatesh Gauri Shankar

#Implementation of a case study project for exploratory data analytics.

Loan Prediction

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

```
data = pd.read_csv("train.csv")
```

```
data.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coapplica
0	LP001002	Male	No	0	Graduate	No	5849	
1	LP001003	Male	Yes	1	Graduate	No	4583	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	
4	LP001008	Male	No	0	Graduate	No	6000	

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID               614 non-null   object
1   Gender                 601 non-null   object
2   Married                611 non-null   object
3   Dependents             599 non-null   object
4   Education              614 non-null   object
5   Self_Employed         582 non-null   object
6   ApplicantIncome        614 non-null   int64
7   CoapplicantIncome      614 non-null   float64
8   LoanAmount             592 non-null   float64
9   Loan_Amount_Term       600 non-null   float64
10  Credit_History         564 non-null   float64
11  Property_Area          614 non-null   object
12  Loan_Status            614 non-null   object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

```
data.apply(lambda x: sum(x.isnull()),axis=0) # checking missing values in each column of train datas
```

Loan_ID	0
Gender	13

```
Married          3
Dependents       15
Education        0
Self_Employed    32
ApplicantIncome  0
CoapplicantIncome 0
LoanAmount       22
Loan_Amount_Term 14
Credit_History   50
Property_Area    0
Loan_Status      0
dtype: int64
```

```
data['Gender'].value_counts()
```

```
Male      489
Female    112
Name: Gender, dtype: int64
```

```
data.Gender = data.Gender.fillna('Male')
```

```
data['Married'].value_counts()
```

```
Yes      398
No       213
Name: Married, dtype: int64
```

```
data.Married = data.Married.fillna('Yes')
```

```
data['Dependents'].value_counts()
```

```
0      345
1      102
2      101
3+      51
Name: Dependents, dtype: int64
```

```
data.Dependents = data.Dependents.fillna('0')
```

```
data['Self_Employed'].value_counts()
```

```
No      500
Yes      82
Name: Self_Employed, dtype: int64
```

```
data.Self_Employed = data.Self_Employed.fillna('No')
```

```
data.LoanAmount = data.LoanAmount.fillna(data.LoanAmount.mean())
```

```
data['Loan_Amount_Term'].value_counts()
```

```
360.0      512
180.0       44
480.0       15
```

```
300.0    13
84.0      4
240.0     4
120.0     3
36.0      2
60.0      2
12.0      1
Name: Loan_Amount_Term, dtype: int64
```

```
data.Loan_Amount_Term = data.Loan_Amount_Term.fillna(360.0)
```

```
data['Credit_History'].value_counts()
```

```
1.0    475
0.0     89
Name: Credit_History, dtype: int64
```

```
data.Credit_History = data.Credit_History.fillna(1.0)
```

```
data.apply(lambda x: sum(x.isnull()),axis=0)
```

```
Loan_ID      0
Gender        0
Married       0
Dependents    0
Education     0
Self_Employed 0
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount    0
Loan_Amount_Term 0
Credit_History 0
Property_Area 0
Loan_Status   0
dtype: int64
```

```
data.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coapplica
0	LP001002	Male	No	0	Graduate	No	5849	
1	LP001003	Male	Yes	1	Graduate	No	4583	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	
4	LP001008	Male	No	0	Graduate	No	6000	

```
# Splitting traing data
X = data.iloc[:, 1: 12].values
y = data.iloc[:, 12].values
```

```
X
```

y

```
# Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 1/3, random_state = 0)
```

X_train

```
array([[ 'Male', 'Yes', '3+', ..., 360.0, 1.0, 'Rural'],
       [ 'Male', 'Yes', '0', ..., 360.0, 1.0, 'Rural'],
       [ 'Male', 'Yes', '3+', ..., 180.0, 1.0, 'Rural'],
       ...,
       [ 'Male', 'Yes', '3+', ..., 360.0, 1.0, 'Semiurban'],
       [ 'Male', 'Yes', '0', ..., 360.0, 1.0, 'Urban'],
       [ 'Female', 'Yes', '0', ..., 360.0, 1.0, 'Semiurban']], dtype=object)
```

```
# Encoding categorical data
# Encoding the Independent Variable
from sklearn.preprocessing import LabelEncoder
labelencoder_X = LabelEncoder()
```

```
for i in range(0, 5):
    X_train[:,i] = labelencoder_X.fit_transform(X_train[:,i])
```

```
X_train[:,10] = labelencoder_X.fit_transform(X_train[:,10])
```

```
# Encoding the Dependent Variable
labelencoder_y = LabelEncoder()
y_train = labelencoder_y.fit_transform(y_train)
```

+ Code

+ Text

X_train

```
array([[1, 1, 3, ..., 360.0, 1.0, 0],
       [1, 1, 0, ..., 360.0, 1.0, 0],
       [1, 1, 3, ..., 180.0, 1.0, 0],
       ...,
       [1, 1, 3, ..., 360.0, 1.0, 1],
       [1, 1, 0, ..., 360.0, 1.0, 2],
       [0, 1, 0, ..., 360.0, 1.0, 1]], dtype=object)
```

y_train

```
array([1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1,
       0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1,
       0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0,
       0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0,
       1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1,
       1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1,
       0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1,
       1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1,
       1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1,
       1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1,
       1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1,
       1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1,
       1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1,
       0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1,
       1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1])
```



```
# Measuring Accuracy
from sklearn import metrics
print('The accuracy of Logistic Regression is: ', metrics.accuracy_score(y_pred, y_test))
```

The accuracy of Logistic Regression is: 0.7073170731707317

```
# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
```

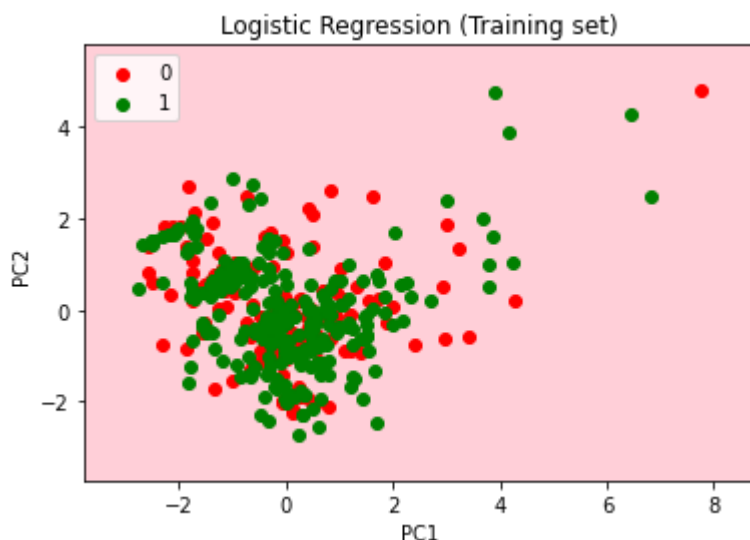
cm

```
array([[ 0, 60],
       [ 0, 145]])
```

```
# Visualising the Training set results
from matplotlib.colors import ListedColormap
X_set, y_set = X_train, y_train
X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:, 0].max() + 1, step = 0.5),
                     np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:, 1].max() + 1, step = 0.5))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('pink', 'lightgreen')))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
                c = ListedColormap(('red', 'green'))(i), label = j)
plt.title('Logistic Regression (Training set)')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.legend()
plt.show()
```

c argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value

c argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value



```
# Visualising the Test set results
from matplotlib.colors import ListedColormap
X_set, y_set = X_test, y_test
```

```

X1, X2 = np.meshgrid(np.arange(start = X_set[:, 0].min() - 1, stop = X_set[:, 0].max() + 1, step = 0.5),
                     np.arange(start = X_set[:, 1].min() - 1, stop = X_set[:, 1].max() + 1, step = 0.5))
plt.contourf(X1, X2, classifier.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape),
             alpha = 0.75, cmap = ListedColormap(('pink', 'lightgreen'))))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i, j in enumerate(np.unique(y_set)):
    plt.scatter(X_set[y_set == j, 0], X_set[y_set == j, 1],
                c = ListedColormap(('red', 'green'))(i), label = j)
plt.title('Logistic Regression (Test set)')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.legend()
plt.show()

```

c argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value

c argument looks like a single numeric RGB or RGBA sequence, which should be avoided as value

